

Harness evaluation of the deed-extraction pipeline

A self-contained summary of what we built, what we tested, what the results mean, what still fails, and what to do next.

1 Executive summary

Purpose. We are building an automated pipeline that reads scanned Odisha land-registration deeds and produces a structured record (deed type, dates, consideration amount, buyers, sellers, property rows). To find out why it fails and whether specific changes help, we built an evaluation harness: a tool that runs each version of the pipeline over a fixed set of deeds, compares the output against a human-validated answer key, and reports where it goes wrong.

Scale. We tested 45 pipeline configurations on a fixed set of 42 deeds (128 pages). Accuracy is reported as F1, a score from 0 to 1 that balances how much of the output is correct against how much of the correct answer is found (higher is better; defined in section 2).

42 deeds / 128 pages	45 configurations tested	0.458 baseline lenient F1	0.518 best observed lenient F1
--------------------------------	------------------------------------	-------------------------------------	--

	Lenient F1	Strict F1
Baseline (unchanged starting pipeline)	0.458	0.316
Best observed configuration	0.518	0.359

Key findings

- **Give the model the whole deed at once.** Splitting a deed into single pages or small chunks made results worse, because related information (the parties and the registration stamp) sits on different pages.
- **Clear, document-specific instructions help.** A worked example plus explicit rules for reading dates and deed numbers raised F1 from 0.458 to 0.497. Generic "be careful" instructions did not help and lowered recall.
- **A targeted second pass fixes a recurring date error.** A separate model call that reads only the last page to find the registration date raised that field from 0.55 to 0.68 F1 (0.73 in the best configuration), correcting cases where the model returned the presentation date instead.
- **Extra effort did not pay off.** Additional model calls, self-verification, voting across repeated runs, and adding page images produced no measurable gain in these tests.
- **The hardest remaining failures are numbers and parties.** Handwritten amounts and plot/khata numbers are often missing or misread in the OCR, and buyers and property rows are frequently not extracted or correctly structured. Fabricated values are rare.

MAIN TAKEAWAY

On this 42-deed set, the baseline scored 0.458 lenient F1. Concrete, document-specific instructions improved the result, and a targeted second pass fixed a recurring date-confusion error.

2 System and evaluation

How the pipeline works. A deed is a scanned, often handwritten document in Odia and English. OCR (optical character recognition) converts each page image into text; that text is labelled by page and passed to Gemma, a language model that reads it and returns the required fields as structured data; the result is one record per deed. The harness then grades that record against a validated answer key.



The first five stages are the pipeline being tested. The final stage is the harness (a separate tool, not part of production), which compares the record field-by-field with the answer key and reports both the score and where each error occurred.

What the harness measures. For every configuration it runs the whole pipeline on all 42 deeds, grades each field, and reports accuracy plus a breakdown of failure causes. The dataset is 42 deeds / 128 pages (2–6 pages per deed), mixing eras (28 from 1985–94, 14 from 1995+). Of 45 configurations, 1 is the baseline and 44 are variants; each was generally run once.

How accuracy is scored.

- **Precision:** of the values the pipeline produced, the share that were correct.
- **Recall:** of the values that should have been found, the share the pipeline actually got.
- **F1:** a single 0-1 score that balances precision and recall.

Strict vs lenient F1. Strict requires a close match after normalising case, punctuation and digits. Lenient additionally forgives harmless differences such as Odia-vs-Latin spelling, honorifics, and number/date formatting. Lenient is the primary score; strict is reported alongside and shows the same ordering at lower values.

BASELINE CONFIGURATION, DEFINED

This is the default configuration of the extraction pipeline (`build_messages.py` / `extract.py`), not the evalkit control that produced the F1 scores in this report (see below). By default it sends every page's image alongside every page's OCR text, interleaved page by page in a single request, with Gemma 31B, temperature 0, JSON mode (schema-guided decoding), and max output 4,096 tokens.

The evalkit harness (`evalkit/experiments.py`) that produced the F1 numbers below instead uses a text-only control (A7_full_boundaries: whole deed's OCR text, concatenated in order with page labels, no page images) as its baseline, testing image-inclusive variants (G1, G3_ocr_1img, G3_ocr_3img) separately on top of it. The two are different pipelines; nothing in this report measures the image-inclusive default above.

2.1 How the harness works: what was varied, covered, combined and counted

What we varied (45 configurations)	Variations tried	Configs
Context & text layout	whole deed vs 1-, 2-, 4- and 8-page chunks; sliding windows; page labels on/off; pages as a JSON list; cleaned or flattened OCR text	9
Prompt	field definitions; worked example; date and deed-number rule; candidate listing; evidence / confidence output; "conservative", anti-hallucination, negative-example and ambiguity wording; step-by-step; OCR-first order; combinations	15
Extra calls	field-by-field (10 calls) and field groups (5); page retrieval (model-selected, page-to-field map, keyword filter); per-page calls with a model merge; last-page registration-date call (3 variants, incl. the best configuration)	9
Verification	verify pass; evidence check; retrieve → extract → verify	3
Images	OCR + 1 image; OCR + 3 images; images only (no OCR text)	3
Voting / repeats	3-sample vote; temperature 0.3; the baseline repeated 3 times (noise check)	5

Plus the baseline (1). Earlier exploratory rounds outside this harness (OCR strip variants, image sizes) used a different scorer and are not counted.

Edge / failure cases covered	What happens
OCR-missing values	Sometimes the correct value simply is not anywhere in the OCR text, so no amount of extraction effort can recover it.
Handwritten numbers	Handwritten amounts and plot/khata numbers are the hardest for OCR to read, especially in the older deeds.
Wrong selection; dates	The model sometimes picks the wrong date, most often confusing the registration date with the presentation date; badly garbled OCR dates cannot be fixed by extraction.
Missing deed number	The deed number was often left blank even when it was present in the OCR; a worked example in the prompt fixed most of these cases.
Multiple buyers / sellers	Deeds with three or more buyers or sellers are the weakest case: the model tends to return only one party instead of all of them.
Extra / mis-structured rows	The model sometimes adds party or property rows the answer key does not list, or splits one real entry into several.
Deed variety	The set covers sale and non-sale deeds (mortgage, gift, exchange) across two eras, but not deeds longer than 6 pages or deeds outside this set.

How good is the OCR?

OCR coverage (63% of 720 validated items) only says a correct value can be *found* in the OCR text. The OCR is also not repeatable: re-running one page gave different text (character similarity 0.345).

Deed no. 100%	Presentation date 88%	Registration date 86%	
Buyer names 70%	Plot nos. 46%	Khata nos. 39%	Amount 26%

How outputs are combined

- **Pages → one input:** a deed's page texts are stacked in order with *PAGE n*: labels into one text; nothing is merged or reconciled.
- **Baseline:** one call returns the whole record; nothing to merge.
- **Best configuration:** two calls on different fields (registration date from the last page; all other fields from the whole deed), joined by field name; no voting.
- **Per-page variation tested:** one call per page, outputs merged by majority vote (single values) and by joining near-duplicate people/plots (lists): F1 0.379 (−0.076). Merging by a further model call: 0.391 (−0.064).

How many model calls per deed?

Configuration	Calls	Configuration	Calls
Baseline (whole deed)	1	Field-by-field	10
Best observed configuration	2	Field groups	5
One page per call (+ merge)	3.0	Verify / evidence check	2
Two-page chunks	1.8	Retrieve → extract → verify	3
4- / 8-page chunks	≈1	3-sample voting	3
Page retrieval (model-selected)	2	Per-page + model merge	4

Mean calls per deed from the experiment records (calls average over deeds; 4-page chunking splits only the 2 deeds longer than 4 pages). Baseline latency 3.8 s, best configuration 4.7 s per deed. Cost in currency was not measured.

3 Experiments and results

Each change was measured against the same fixed baseline, deed-by-deed. Because repeating the baseline alone varied by about ±0.01 F1, differences smaller than that are not interpretable.

Question tested	What changed	Result	Interpretation
Do clearer instructions help?	A worked example plus explicit date and deed-number rules (improved instructions)	F1 0.458 → 0.497 (+0.038)	Gains concentrate in the fields the rules address (deed numbers, dates). Generic caution instead lowered recall.
Can the recurring date error be fixed?	A separate call reading only the last page to extract the registration date (targeted registration-date extraction)	F1 0.505 (+0.046); date field 0.55 → 0.68 (0.73 in best config)	Of 19 wrong dates in the baseline, 13 were the presentation date, which repeats on earlier pages. Isolating the last page removes the confusion.
Does listing candidates before choosing help?	The model lists candidate values and supporting text, then selects (candidate-listing variant)	In best config F1 0.518; added step alone within noise	Highest point estimate, but its separate contribution is not demonstrated and it costs more tokens and latency.

Whole deed or page chunks?	Split the deed into single pages or small chunks instead of one request	Single page -0.076; two-page chunks -0.038	Parties and the registration stamp sit on different pages; splitting separates information that must be read together. (4- and 8-page chunks equal the baseline here: no deed is longer than 6 pages.)
Do more calls or self-verification help?	Field-by-field calls, page retrieval, a verify pass, an evidence check	All within noise; a per-page merge was worse (-0.064)	The model's quoted "evidence" is copied OCR text even when the chosen value is wrong, so verification cannot catch these errors.
Do images or voting help?	Add page images to the text; or sample three times and vote	Images +0.006 / +0.010; images-only -0.191; voting $\approx$ single run	Errors are systematic (the same wrong value each time), so resampling does not remove them; images added no benefit and, used alone, were far worse.

BEST OBSERVED CONFIGURATION

It combines the two supported improvements (improved instructions and targeted registration-date extraction) with the candidate-listing variant. Together they reach lenient F1 0.518 (+0.058 over baseline), better on 29 of 42 deeds. Only the first improvement (instructions) is clearly beyond the noise; the later additions are small and not individually proven, so the configuration is a best-observed candidate rather than a demonstrated recipe.

4 Error analysis

Every incorrect or missing value was traced to one of a few causes. The counts below are for the baseline (612 incorrect items in total) and show where the problems concentrate.

Failure source	Count	What it means
Value missing from the OCR text	199	The OCR never captured it, usually a handwritten number. No extraction change can recover it.
Present in OCR but not extracted	152	The value was in the text, but the model returned nothing. An extraction gap, not an OCR problem.
Wrong value selected	91	Several candidates were present and the model picked the wrong one (e.g. presentation date instead of registration date).
Extra or mis-structured party/property row	166	The model produced a row the answer key does not list, or split one real entry into several.

Fabricated values (not found anywhere in the OCR) were rare: 3 of 612. Two separate bottlenecks dominate: OCR (handwritten numbers never captured) and extraction/structure (values present but missed or mis-grouped).

Field-level findings

Field	What the evidence shows
Registration date	Improved most, 0.55 $\rightarrow$ 0.68 via the targeted last-page call (0.73 in the best configuration). Ceiling: the correct date is present in the last page's OCR for only 34 of 42 deeds.
Consideration amount	Weak ($\approx$ 0.32–0.35) and largely OCR-bound: the correct amount appears in the OCR for only 26% of deeds (14% in the older era). The model copies what the OCR holds: an OCR problem, not fabrication.
Buyers	Very low ($\approx$ 0.16) even though buyer names are present in the OCR for 70% of rows. In multi-buyer deeds the model typically returns only one buyer: an extraction and table-reading problem.
Handwritten numeric fields	Amounts and plot/khata numbers are the clearest OCR bottleneck, especially in older (1985–94) deeds, where numeric coverage falls as low as 14–32%.

5 Conclusion

What the results support

- Give the model the whole deed in one request; do not split small deeds into pages.
- Use concrete, document-specific instructions (a worked example plus date and deed-number rules); avoid generic caution, which lowers recall.
- Use a targeted last-page call for the registration date to fix the date-confusion error.
- Do not expect gains from extra calls, self-verification, voting, or whole-page images on this task.